Dice Throws

AlgoExpert · Hard · Dynamic Programming

The problem

- Number of ways to roll numDice dice with numSides sides to total `target`.

Intuition

- Count ways to reach each total with dice added one at a time.

Brute force vs optimal

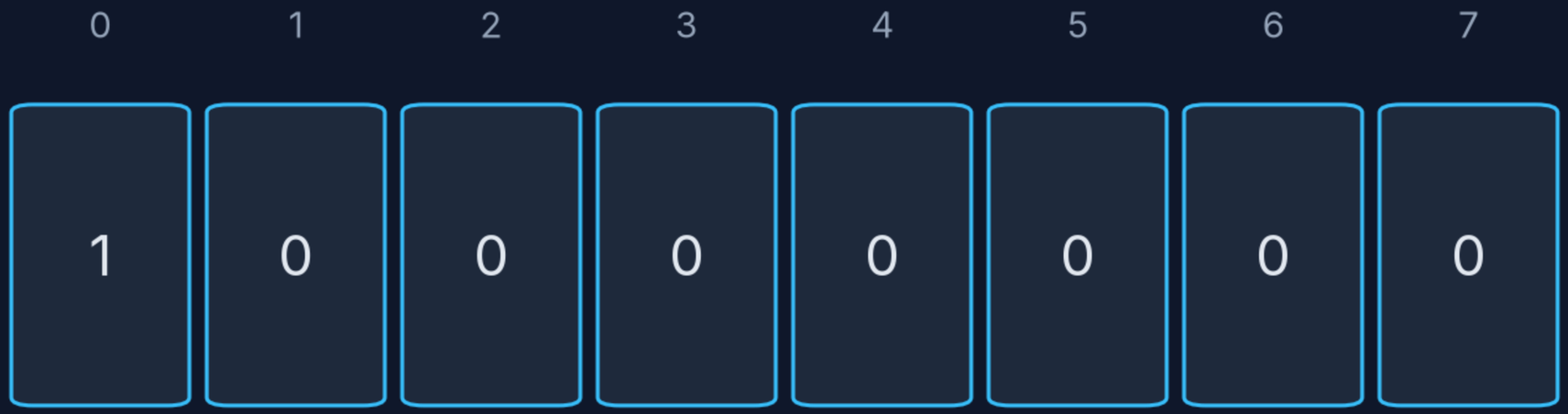
- Recurse over each die's face → O(sides^dice)
- DP over (dice, total) → O(dice · target · sides)
- Rolling array

Approach

- 1. ways for total zero with no dice is one
- 2. For each die, roll every face
- 3. Shift counts forward
- 4. Read ways of target

Worked example

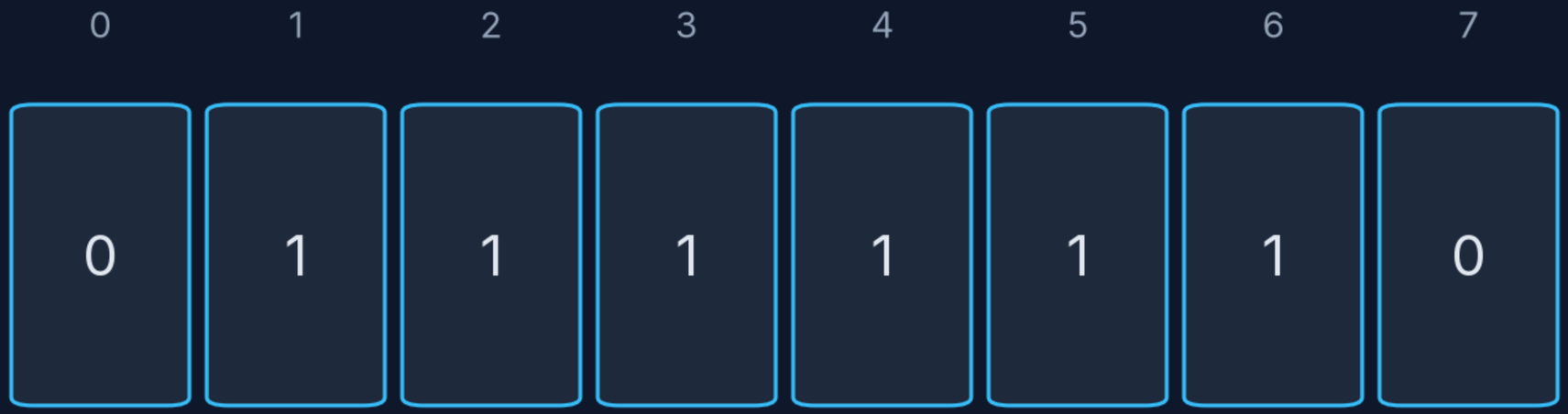
ways to reach each total (0 dice)



0 dice: one way to make 0

Worked example

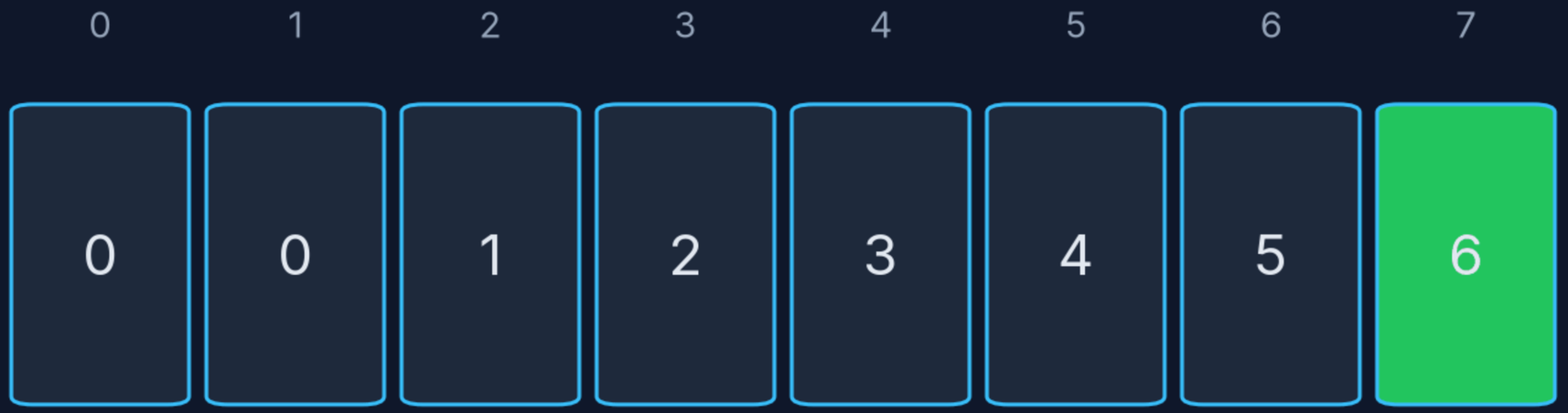
ways after 1 dice



After die 1: ways[7] = 0

Worked example

ways after 2 dice



After die 2: ways[7] = 6

Java solution

```java
public class DiceThrows {
    public static int diceThrows(int numDice, int numSides, int target) {
        long[] ways = new long[target + 1];
        ways[0] = 1;
        for (int d = 0; d < numDice; d++) {
            long[] next = new long[target + 1];
            for (int t = 0; t <= target; t++) if (ways[t] != 0)
                for (int s = 1; s <= numSides && t + s <= target; s++) next[t + s] += ways[t];
            ways = next;
        }
        return (int) ways[target];
    }
}
```

Complexity

- Time: O(d * t * s)
- Space: O(t)

Edge cases & pitfalls

- Target out of range → 0
- Use long / mod for big counts
- Skip zero entries

Interview tips & follow-ups

- Follow-up: probability = ways / sides^dice

Recap

- Count ways to reach each total with dice added one at a time.
- Complexity: O(d * t * s)